

Production Implementation Plan: Cyber Threat Intelligence MVP

An Entrepreneurial Blueprint for a High-Value Serverless Vulnerability Intelligence SaaS

This deployment plan outlines the end-to-end engineering, pipeline logic, and deployment strategy for building an automated, AI-powered daily vulnerability intelligence and alerting system. The architecture focuses heavily on high scalability, low-latency execution, and zero operational overhead during the validation phase.

1. Technical Stack Architecture

The system utilizes a modern, entirely serverless decoupled architecture designed to eliminate fixed infrastructure overhead while ensuring high structural durability and speed.

Layer	Technology Selection	Strategic Purpose & Advantage
Frontend UI	Next.js (App Router) & Tailwind CSS	Enables rapid design iteration. Hosts the Landing Page, Client Onboarding flow, and User Profile configuration panel on Vercel Edge networks.
Backend Automation	Cloudflare Workers / Supabase Edge Functions	Executes distributed scheduled programmatic ingestion loops and parsing cycles entirely at edge nodes using Cron triggers without maintaining live application server runtimes.
Database Layer	Supabase Cloud (PostgreSQL)	Provides native Row-Level Security (RLS), structural SQL querying capabilities, and advanced JSONB indexing mechanisms for polymorphic vulnerability data schemas.
Email Delivery	Resend API	High-reputation dynamic transactional delivery framework with granular TypeScript library integration to dispatch highly stylized HTML digests directly to user target inboxes.

2. Source Data Ingestion Strategy

The orchestration framework initiates an automated ingestion loop every 24 hours to aggregate and normalize threat vectors from three public security data registries:

- **National Vulnerability Database (NVD):** Connects to the official REST service endpoints utilizing the tracking filters *lastModStartDate* and *lastModEndDate* to fetch the absolute latest batch of newly classified or altered Common Vulnerabilities and Exposures (CVEs).
- **CISA Known Exploited Vulnerabilities (KEV) Catalog:** Ingests the authoritative real-world tracking matrices to determine if any newly recorded CVEs are actively being weaponized by threat actors in live production environments.

- **FIRST Exploit Prediction Scoring System (EPSS):** Pulls from the probabilistic modeling API to ascertain the likelihood of any newly uncovered exploit attempt occurring across the wider cyber ecosystem within the subsequent 30 days.

3. Parsing & Scoring Logic

To systematically eliminate alert fatigue, the pipeline processes the composite data arrays and subjects them to a threat-informed triage algorithm that weights actual risk vectors significantly higher than purely academic or theoretical ones.

$$\text{Priority Score} = (\text{CVSS Base} \times 0.4) + (\text{EPSS Probability} \times 10 \times 0.3) + (\text{CISA KEV Status [0 or 1]} \times 3.0)$$

Pipeline Execution & Personalization Flow

1. **Data Normalization:** Raw CVE entities are scraped, transformed, and mapped into clean relational profiles, standardizing messy vendor identities (e.g., standardizing strings like "Cisco Systems Inc." or "cisco" into a singular uniform reference).
2. **User Intersection Mapping:** The execution worker retrieves all registered active user accounts from the database along with their custom infrastructure technology stack preference tags.
3. **The "Top 10" Isolation Loop:** For every subscriber, the logic cross-references their preference matrix against the parsed 24-hour vulnerability cache, sorts matching elements descending by the calculated *Priority Score*, and extracts exactly the top 10 rows.
4. **Global Threat Backfilling:** In scenarios where a customer's chosen technical footprint yields fewer than 10 total alerts, the engine fills the remaining open slots with the highest-scoring global threat entries to ensure a full value-packed summary is provided.

4. Enterprise AI Enhancement Framework

The AI operations engine runs context-enrichment actions directly inside the data ingestion microservice rather than on a per-user delivery basis. This design caches computed outputs within PostgreSQL, keeping API token costs at near-zero expenditures.

AI Engine Cost-Optimization Strategy

If 2,000 distinct enterprises all track an identical underlying operating system dependency, the LLM processes the source CVE text **exactly once** during the initial database storage step. The resulting clean plain-English summary and command-line execution workarounds are dynamically populated across every matching user report instantly.

Targeted Prompt Engineering Blueprints

The Technical-to-Business Risk Translator:

System: You are an elite Cyber Threat Intelligence analyst. Translate complex, academic vulnerability parameters into brief, high-impact functional operational summaries for

corporate IT administrators.

User: Translate this raw vulnerability source object: [Insert Target CVE JSON Data Array]

Output: Limit response to two plain English sentences. State precisely what risk an adversary poses to operations and the functional business infrastructure impact. Do not print out the baseline CVE reference ID.

The Automated Patch Remediation Engine:

System: You are an automated systems engineering supervisor tasked with infrastructure recovery.

User: Output the explicit commands needed to patch the following risk element: [Insert Target CVE Structural Architecture Payload]

Output: Produce only a highly optimized clean Markdown bash shell code block snippet or a precise config parameter change. Keep context notes to absolute zero.

5. Dynamic Email Generation & Delivery

The dispatch service generates micro-targeted responsive HTML templates that load straight to mobile clients without rendering latency or clipping.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: -apple-system, sans-serif; background-color: #f8fafc; color:
#0f172a; }
    .card { background: #ffffff; border: 1px solid #e2e8f0; border-radius: 6px; padding:
16px; margin-bottom: 12px; }
    .badge-kev { background-color: #fee2e2; color: #991b1b; font-size: 11px; font-weight:
bold; padding: 2px 6px; border-radius: 4px; }
    .code-block { background-color: #f1f5f9; font-family: monospace; padding: 10px;
border-radius: 4px; }
  </style>

<body>
  <h2>Your Custom Top 10 Vulnerability Digest</h2>
  <p>Tailored precisely to your infrastructure configurations.</p>

  <div class="card">
    <h3>CVE-2026-3812: Critical Remote Code Execution <span class="badge-kev">Active
Exploitation</span></h3>
    <p><strong>Risk Assessment:</strong> Remote attackers can bypass authentication on
your public gateways to run unauthorized processes, posing a severe risk of complete
infrastructure compromise.</p>
    <h4>🔧 Remediation Playbook</h4>
    <div class="code-block">sudo apt-get update && sudo apt-get --only-upgrade install
linux-image-generic</div>
  </div>
</body>
</html>
```

6. Production Database Schema

Execute this foundational schema script in the Supabase SQL editor to create the core database structure, index patterns, and access control policies for your MVP:

```
-- Enable UUID extension for secure, non-sequential system identifiers
create extension if not exists "uuid-oss";

-- 1. Client Profiles Configuration Metadata Table
create table public.profiles (
  id uuid references auth.users on delete cascade primary key,
  email text unique not null,
  tech_tags text[] default '{} '::text[] not null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- 2. Daily Master Vulnerability Ingestion & Intelligence Cache Table
create table public.vulnerability_cache (
  id text primary key, -- Stores the explicit reference identification (e.g.,
  'CVE-2026-1042')
  title text,
  vendor text not null,
  product text not null,
  cvss_score numeric(3,1),
  epss_score numeric(5,4),
  is_cisa_kev boolean default false not null,
  priority_score numeric(5,2) not null,
  ai_summary text,
  ai_playbook text,
  published_at timestamp with time zone,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- Establish database performance indices for large scale batch iterations
create index idx_vuln_priority on public.vulnerability_cache (priority_score desc);
create index idx_vuln_vendor_product on public.vulnerability_cache (vendor, product);
create index idx_profiles_tags on public.profiles using gin (tech_tags);

-- Activate database structural Row-Level Security (RLS) safeguards
alter table public.profiles enable row level security;
alter table public.vulnerability_cache enable row level security;

-- Enforce client security barriers
create policy "Users can view own profile data." on public.profiles
  for select using (auth.uid() = id);

create policy "Users can modify own tags." on public.profiles
  for update using (auth.uid() = id);

create policy "System cache is viewable by verified users." on public.vulnerability_cache
  for select using (auth.role() = 'authenticated');
```

7. Lifecycle Deployment Roadmap

1. **Database Provisioning:** Initialize the target Supabase Postgres workspace and apply the relational table schema definitions and configuration parameters detailed above.
2. **Microservice Setup:** Package and dispatch the ingestion scripts to Cloudflare Workers using the wrangler command-line suite (*wrangler deploy*).
3. **Scheduling Implementation:** Attach a Cloudflare CRON event trigger targeting a daily synchronization cycle exactly at *02:00 UTC*.
4. **UI Setup & Hooks:** Bind the Next.js software repository to Vercel to establish seamless main-branch continuous deployment pipelines, and configure the necessary production environment access tokens for Supabase and the Resend API.